

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION COMMUNICATION TECHNOLOGY



SOICT

PROJECT REPORT:

Steam Game Analysis Data Pipeline

Supervised by:

Ph.D. Tran Viet Trung

Presented by:

Vu Tuan Truong - 20225535

Nguyen Hoang Son Tung - 20225536

Ngo Duy Dat - 20225480

Nguyen Nhat Minh - 20225510

Doi Sy Thang - 20225528

Hanoi - Vietnam

2025

Contents

1	Introduction	3
1.1	Problem Context	3
2	Architecture and Design	3
2.1	Overall Architecture	3
2.2	Detailed Components	5
2.2.1	Ingestion Layer	6
2.2.2	Processing Layer	6
2.2.3	Storage Layer	6
2.2.4	Serving Layer	7
2.3	Data Flow	7
2.3.1	Ingestion Flow: From Source to Stream	7
2.3.2	Processing Flow: From Stream to Insight	7
2.3.3	Storage Flow: The Dual-Sink Pattern	8
3	Implementation Details	8
3.1	Infrastructure Implementation	8
3.1.1	Physical Cluster and Networking	8
3.1.2	Kubernetes with k3s	8
3.1.3	Deployment Orchestration with Helm	9
3.2	Data Mining and Ingestion	9
3.2.1	Directory Structure	9
3.2.2	Data Mining Phase (Historical Acquisition)	10
3.2.3	Data Preparation and Bootstrap Phase	11
3.2.4	State Management and Deduplication (Redis)	12
3.2.5	Real-Time Ingestion Phase (Producers)	12
3.2.6	Kafka Infrastructure Configuration	14
3.3	Spark Processing Layer	15
3.3.1	Silver Layer: Data Cleaning & Ingestion	15

3.3.2	Gold Layer: Aggregation & Analytics	16
3.4	Distributed Learning Pipelines with Spark ML	19
3.4.1	Regression Pipeline for Game Playtime Prediction	20
3.4.2	Recommendation Pipeline for Implicit Feedback Modeling	20
3.5	Monitoring and Quality Assurance	21
3.5.1	Data Quality Strategy: Quarantine and Audit	21
3.5.2	System Observability	22
4	Lessons Learned	26
4.1	Lesson 1: Strategic Planning	26
4.1.1	Problem Description	26
4.1.2	Approaches Tried	26
4.1.3	Final Solution	26
4.1.4	Key Takeaways	27
4.2	Lesson 2: The "Small Files" Problem in Streaming	27
4.2.1	Problem Description	27
4.2.2	Solution	27
4.3	Lesson 3: Optimization Strategies in Spark Structured Streaming	27
4.3.1	Problem Description	27
4.3.2	Approaches Tried	28
4.3.3	Final Solution	28
4.3.4	Key Takeaways	28
4.4	Lesson 4: Resource Management on Commodity Hardware	28
4.4.1	Problem Description	28
4.4.2	Solution	29
4.4.3	Key Takeaways	29

1 Introduction

1.1 Problem Context

The digital game distribution market, dominated by platforms like Steam, generates vast amounts of real-time data spanning player behavior, commercial performance, and community sentiment. This data is invaluable for developers, publishers, and analysts seeking to understand market trends, optimize pricing strategies, and engage with their communities. However, the data landscape is fragmented across various endpoints—the Steam Web API provides core metadata, the Storefront offers pricing and visual assets, SteamSpy estimates ownership figures, and SteamCharts tracks concurrent player concurrency. Furthermore, this data arrives at vastly different velocities: game releases are sporadic, player counts fluctuate minute-by-minute, and reviews pour in continuously during viral events.

This heterogeneity poses a significant engineering challenge: data must be collected from disparate sources, normalized into a consistent schema, and delivered reliably to downstream analytical engines. Traditional batch-only pipelines suffer from high latency, often delivering insights days after critical events occur. Conversely, stream-only pipelines lack the historical context needed to identify long-term trends.

This project addresses these challenges by building a unified, scalable data pipeline capable of ingesting both deep historical baselines and high-velocity real-time events. The specific focus of this report is the **Data Mining and Ingestion Layer**, which serves as the critical entry point for the entire system. The core objectives for this layer are:

1. **Historical Completeness:** Recovering full game catalogs, extensive review histories for top titles, and complete player count trends since launch to establish robust analytical baselines.
2. **Real-Time Responsiveness:** Capturing new game releases, fresh user reviews, and player concurrency changes with seconds-to-minutes latency to enable rapid reaction to market shifts.
3. **Unified Ingestion:** Normalizing diverse data sources into a single, immutable event log using Apache Kafka, ensuring data consistency and simplifying downstream consumer logic.
4. **Resilience and Scalability:** Implementing robust error handling, rate limiting, and state management to operate reliably in a distributed Kubernetes environment across multiple nodes.

2 Architecture and Design

2.1 Overall Architecture

Adopting the **Kappa Architecture**, our system streamlines big data processing by treating all inputs as an infinite, immutable stream. Unlike traditional Lambda architectures that maintain separate codebases

for batch and real-time layers, this approach utilizes a single streaming engine—Apache Spark Structured Streaming—to handle both live data ingestion and historical reprocessing. The central backbone of this architecture is the Apache Kafka log, which serves as the single source of truth for all downstream analytics.

Within this streaming context, we implement a **Medallion Architecture** to structure data evolution:

- **Bronze Layer (Raw):** Represented directly by the **Kafka topics**, this layer retains the raw, immutable event log exactly as ingested from our scrapers. By persisting data here first, we ensure a replayable history without needing a separate raw file storage bucket.
- **Silver Layer (Refined):** As data streams out of Kafka, Spark jobs clean, deduplicate, and enforce strict, typed schemas before writing to **Iceberg tables**. This conversion turns semi-structured JSON streams into reliable, ACID-compliant tabular data suitable for analytical queries.
- **Gold Layer (Aggregated):** The final processing stage aggregates these refined streams into business-ready metrics. These insights are calculated in real-time and persisted using a **Dual-Sink Pattern**: simultaneously updating Iceberg tables for long-term historical analysis and a MongoDB document store to power low-latency operational dashboards.

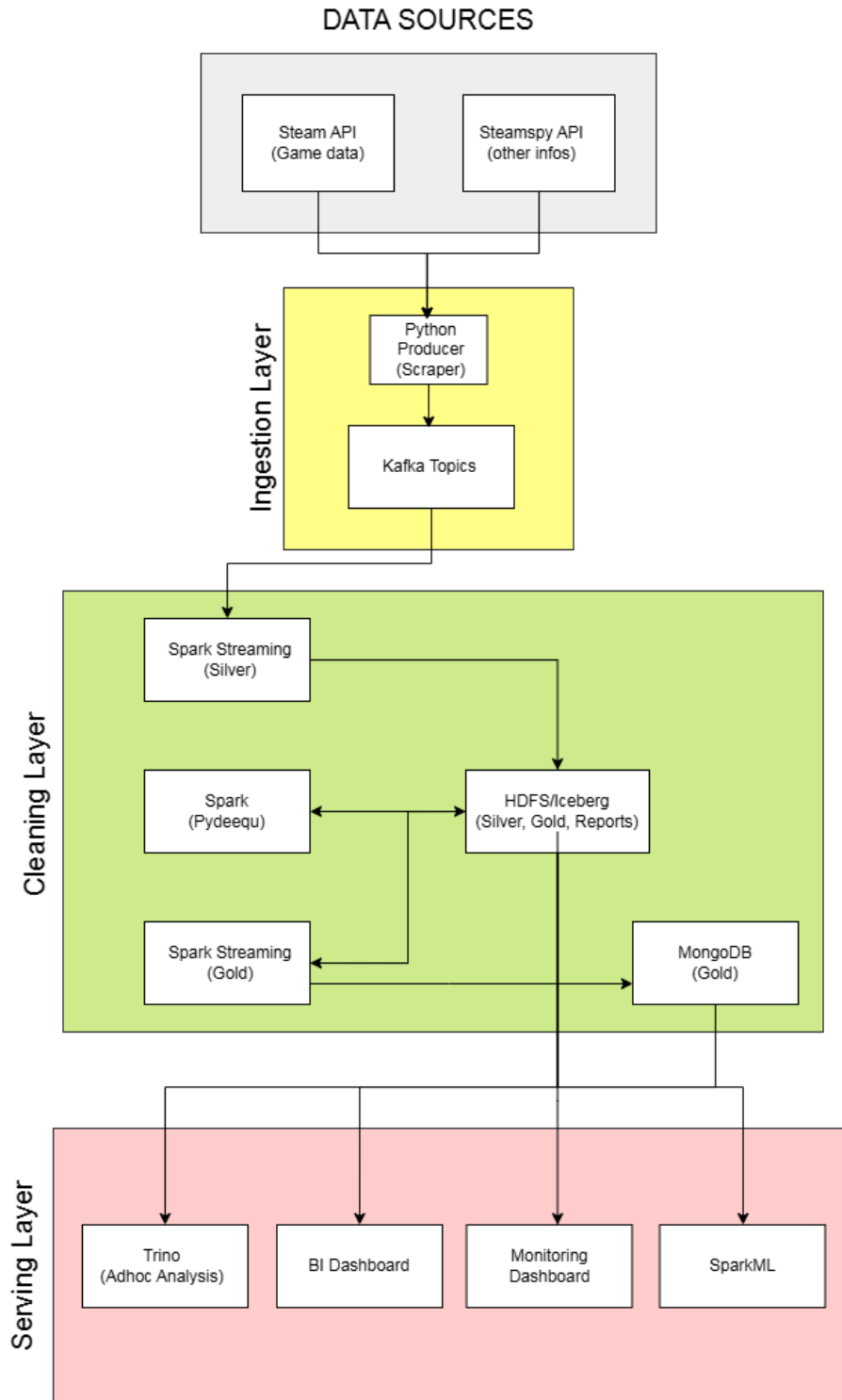


Figure 1: Overall system architecture based on Kappa and Medallion patterns

2.2 Detailed Components

To realize the Kappa architecture on a distributed Kubernetes infrastructure, we selected a stack of cloud-native technologies that prioritize horizontal scalability and automated operations. The system's

logical layers are mapped to specific components as follows:

2.2.1 Ingestion Layer

This layer is responsible for the reliable acquisition and buffering of high-velocity data.

- **Strimzi Kafka Operator:** We deploy Apache Kafka within Kubernetes using the Strimzi operator. This “Kubernetes-native” approach simplifies the management of stateful brokers, automating tasks like rolling updates and rebalancing. Kafka topics serve as the persistent, immutable log for all incoming data.
- **Python Micro-Producers:** Custom ingestion services (built as Dockerized Python applications) actively poll the Steam Web API and scrape store pages. To handle the scale of Steam’s catalog efficiently, these producers utilize **Redis** as a high-speed state store—managing de-duplication sets and crawl frontiers to ensure we only ingest novel or updated data points.

2.2.2 Processing Layer

All data transformation logic resides here, unifying batch and stream processing.

- **Spark on Kubernetes:** We utilize the **Spark Operator** to manage the lifecycle of Spark applications as custom Kubernetes resources (CRDs). This allows us to submit, monitor, and scale Spark driver and executor pods using standard `kubectl` tooling.
- **Structured Streaming:** The core logic is implemented using Spark Structured Streaming. This engine handles the continuous transformation of raw JSON from Kafka into typed Silver tables and complex Gold aggregates, managing stateful window operations (e.g., 5-minute tumbling windows for review velocity) and late-arriving data automatically.
- **PyDeequ:** To maintain the integrity of our Data Lake, we implement a "Promotion Gate" pattern. Data landing in the Silver layer immediately undergoes a battery of quality tests defined in PyDeequ. Only records that pass critical validations (e.g., referential integrity, schema compliance) are promoted to the production tables via a merge operation, while any failing records are side-lined into a quarantine audit log for developer inspection.

2.2.3 Storage Layer

A modern Data Lakehouse foundation ensures data is both cheap to store and easy to query.

- **HDFS (Hadoop Distributed File System):** Provides the underlying distributed object storage. We deploy HDFS across the cluster’s nodes to create a unified reliable storage layer on top of commodity hardware.

- **Apache Iceberg:** We overlay the Iceberg table format on HDFS to provide ACID transaction guarantees. Iceberg allows concurrent writing by Spark jobs and reading by analytical engines without lock contention, solving the "small files" problem inherent in raw streaming writes.

2.2.4 Serving Layer

To satisfy diverse consumption patterns, we employ a hybrid serving strategy.

- **MongoDB:** Functions as the fast operational store. Our Spark jobs push the latest "Gold" insights here to support near-real-time dashboards that require sub-second lookup latencies.
- **Trino:** A distributed SQL query engine connected to the Iceberg catalog. It enables ad-hoc data discovery and complex analytical queries over the entire historical dataset without needing data movement.

2.3 Data Flow

The system's data processing pipeline operates as a continuous loop, ensuring that new game information and player interactions are reflected in our analytics within minutes of their occurrence. This flow is divided into three distinct stages:

2.3.1 Ingestion Flow: From Source to Stream

The journey begins with our Python micro-producers, which orchestrate the data crawl. For the game catalog, the producer consults **Redis** to check two sets: `steam:apps:scraped` (apps already processed) and `steam:apps:to_scrape` (the frontier queue). This stateful coordination prevents redundant API calls and ensures we respect Steam's rate limits.

Once a new game or review is fetched, the payload is serialized into JSON and published immediately to the corresponding Kafka topic (e.g., `steam-games`, `steam-reviews`). This decoupling ensures that temporary spikes in data volume are buffered by Kafka, protecting downstream consumers from being overwhelmed.

2.3.2 Processing Flow: From Stream to Insight

Apache Spark subscribes to these Kafka topics using Structured Streaming, reading data in micro-batches.

1. **Silver Promotion:** Raw JSON data is initially landed in a staging table (`steam_games_landing`). The data quality job then scans this landing area, executing PyDeequ verification suites. Valid records are atomically promoted to the authoritative Silver tables using an upsert (`MERGE INTO`) strategy, ensuring that the main analytical tables contain only high-quality, deduplicated data.

2. **Gold Aggregation:** Concurrently, Spark computes business metrics over these streams. For instance, review sentiment is aggregated using 5-minute tumbling windows to calculate real-time “review velocity” and “sentiment ratios,” which are crucial for detecting sudden review bombing events.

2.3.3 Storage Flow: The Dual-Sink Pattern

To serve both our analytical and operational needs without doubling the processing cost, we implement a **Dual-Sink** pattern at the end of our Spark jobs. Using the `foreachBatch` output mode, each micro-batch of aggregated Gold data is written to two destinations simultaneously:

- **To Data Lake (Iceberg):** Rows are appended to the historical Gold tables, available for long-term trend analysis via Trino.
- **To Operational Store (MongoDB):** The same insights are upserted into MongoDB collections. This ensures that the live dashboard always reads the most current state (e.g., "Top 10 Trending Games right now") without needing to scan heavy historical files.

3 Implementation Details

3.1 Infrastructure Implementation

One of the defining characteristics of this project is its deployment on a truly distributed, commodity hardware cluster. Rather than using cloud services, we assembled a physical cluster from 3-5 personal laptops connected over a WAN, simulating a realistic on-premises or edge computing scenario.

3.1.1 Physical Cluster and Networking

The cluster nodes run Ubuntu 22.04 under WSL2 (Windows Subsystem for Linux), providing a near-native Linux environment. To form a cohesive network across disparate physical locations and NAT boundaries, we employ **Tailscale**, a modern WireGuard-based mesh VPN. Tailscale creates a flat, encrypted network overlay, allowing every node to address every other node via stable IPs regardless of their underlying network topology. This is critical for distributed systems like Kafka and HDFS, which rely on consistent peer-to-peer communication.

3.1.2 Kubernetes with k3s

For container orchestration, we chose **k3s**, a lightweight Kubernetes distribution optimized for resource-constrained environments. Unlike standard Kubernetes, k3s bundles its components into a single binary, significantly reducing memory footprint and operational complexity. The k3s cluster was initialized with Tailscale’s VPN authentication, ensuring that agent nodes could securely join the cluster over

the encrypted mesh. This setup allows participants to contribute their machines as worker nodes simply by running a single command.

3.1.3 Deployment Orchestration with Helm

Managing the lifecycle of complex distributed systems like Kafka and HDFS is handled by **Helm**, the Kubernetes package manager. We define custom `values.yaml` files to tailor the behavior of community-maintained charts. Key deployments include:

- **Strimzi Kafka Operator:** Deploys and manages the Kafka cluster as a set of CRDs.
- **Hadoop HDFS:** Deploys NameNode and DataNode pods with persistent storage.
- **Spark Operator:** Manages the submission and lifecycle of Spark applications.
- **MongoDB:** Provides the operational serving store, deployed via the Bitnami chart.

Custom application manifests (for producers and Spark jobs) are stored in the `deploy/` directory and applied using standard `kubectl` commands, providing full version control over our infrastructure.

3.2 Data Mining and Ingestion

This section provides a comprehensive deep dive into the technical implementation of the data gathering and ingestion subsystem, structured by the data lifecycle phases.

3.2.1 Directory Structure

The codebase is organized into modular directories distinguishing between one-off scripts, shared utilities, and continuous services.



Figure 2: Project Directory Structure for Ingestion Layer

3.2.2 Data Mining Phase (Historical Acquisition)

The data acquisition strategy employs specialized scrapers tailored to the unique constraints and formats of each data source. A core design decision in this phase was the use of **JSONL (JSON Lines)** as the intermediate storage format. Unlike CSV, JSONL handles the nested, schema-less nature of API responses (like list-type fields for "Developers" or "Tags") natively without complex serialization overhead, and it supports append-only writing which is crucial for recoverability.

Master Catalog Scraper The `scrape_all_apps.py` script is responsible for building the master dimension table of all available Steam games.

- **Source Aggregation:** It combines data from three distinct sources: the Steam Web API (`ISteamApps/GetAppList`) for the master ID list, the Steam Store API (`appdetails`) for rich metadata like pricing and genres, and the SteamSpy API for owner estimates and playtime stats.
- **Data Richness:** Beyond basic metadata, this scraper specifically targets the "User Tags" and "Categories" fields. These community-driven labels (e.g., "Souls-like", "Cyberpunk") are critical for the downstream clustering algorithms in the Analytics Layer.
- **Resilience Strategy:** Given the scale of 190,000 endpoints, API rate limits are a certainty. The script implements a robust exponential backoff algorithm (retrying with increasing delays of 1.5s, 3s, 6s) to gracefully handle HTTP 429 (Too Many Requests) responses without crashing.
- **Resource Optimization:** To prevent memory overflow during long scraping sessions, the script implements batch writing. It flushes data to the disk every 500 records, ensuring a constant, low memory footprint regardless of the dataset size.

Review Backfill Scraper The `scrape_reviews.py` script mines high-volume textual data for sentiment analysis.

- **Strategic Targeting:** Mining reviews for every game is computationally infeasible and analytically noisy. The script is configured to target the top 10 high-traffic games (e.g., PUBG, CS2, Wukong), which account for the vast majority of user engagement.
- **Cursor Pagination:** The Steam Review API uses a token-based cursor system for pagination. The script intelligently navigates this system to fetch complete review histories, often traversing tens of thousands of pages for a single game.
- **State Persistence Resumability:** Scraping millions of reviews takes days. The script persists the last successful `cursor` token to a local JSON state file (`scraping_state.json`) after every batch. This critical feature allows the process to crash or be stopped and resume exactly where it left off, preventing data loss or duplication.

Historical Player Count Miner The `backfill_player.py` script reconstructs the complete history of concurrent player trends since a game's launch.

- **Technique:** SteamCharts.com is protected by Cloudflare and WAFs. Standard HTTP libraries fail to access it. This script utilizes `undetected_chromedriver` (running Chrome v131) to simulate a real user browser session, successfully bypassing anti-bot protections.
- **Parsing Normalization:** It navigates to each game's page, parses the "Monthly" history table using BeautifulSoup, and transforms the unstructured HTML into a standardized JSON schema containing `event_timestamp`, `avg_players`, and `peak_players`, enabling unified downstream processing.

3.2.3 Data Preparation and Bootstrap Phase

Before the data enters the production pipeline, it undergoes rigorous preparation and initialization to ensure quality and performant ingestion.

Filtering Utility (The Pareto Principle) Resource constraints on the test cluster, combined with the "shovelware" problem on Steam (where thousands of games have zero players), necessitate dataset optimization. The `filter_top_games.py` utility addresses this by applying the Pareto Principle.

- **Smart Parsing:** It handles complex, non-standard fields like `steamspy_owners` (e.g., "20,000..50,000" or "2,000,000+") to derive numerical sort keys.
- **Impact:** By filtering the catalog to the top 10,000 games by ownership, we retain the commercially significant market segment (likely capturing >95% of total revenue and playtime) while reducing the Kafka storage footprint from ~5GB to ~500MB. This optimization is crucial for maintaining high throughput on limited hardware.

Topic Bootstrap Loader The `bootstrap_data.py` script is the bridge between static files and the streaming log.

- **Clean Slate:** It performs an idempotent "reset" operation (Delete → Create) on the Kafka topics, ensuring the pipeline starts with a known clean state free of duplicate or corrupt data.
- **Chronological Replay:** This is the key to the Kappa Architecture. The script reads historical JSONL files, sorts them strictly by `event_timestamp`, and publishes them to Kafka in order. This simulates a "replay" of history, allowing the exact same streaming logic to process 10 years of history as if it were happening in real-time.

3.2.4 State Management and Deduplication (Redis)

In a distributed microservices environment where producers are stateless (pods die and restart), maintaining a persistent "memory" of what has been processed is critical. We establish a dedicated **Redis Layer** to serve as the coordination ledger for the entire ingestion pipeline.

Design Strategy Redis was chosen over a relational database for this role due to its support for **Set** data structures, which offer $O(1)$ time complexity for membership checks. When checking if a specific Game ID is new against a set of 190,000 existing IDs, Redis performs consistently in microseconds, whereas checking against a SQL index or scanning a list would degrade as the dataset grows.

State Initialization The process begins with `bootstrap_redis.py`. Before any live producers are deployed, this script pre-populates the Redis state from the static bootstrap files.

- **Cold Start Prevention:** If we launched the producers without this step, they would see an empty state and attempt to re-scrape and re-ingest the entire history (the "Cold Start" problem), causing massive data duplication in Kafka.
- **Seed Data:** It populates `steam:apps:scraped` with the 10,000 top IDs and seeds the review deduplication sets.

Key Schema The system relies on a strictly defined key schema to manage coordination:

Redis Key Pattern	Type	Purpose
<code>steam:apps:scraped</code>	Set	Global Registry. Contains the IDs of every game ever ingested. Used by Game Catalog Producer to calculate Δ_{new} .
<code>steam:reviews:{id}:recent_ids</code>	Set	Sliding Window. Contains IDs of reviews seen in the last hour for a specific App ID. Used by Live Review Producer.
<code>steam:apps:discarded</code>	Set	Negative Cache. IDs known to be invalid or inaccessible, preventing repeated failed API calls.

Table 1: Redis State Management Schema

3.2.5 Real-Time Ingestion Phase (Producers)

The live ingestion layer consists of three specialized Python microservices, each running as a Deployment in the Kubernetes cluster. These producers share a common "robust loop" design pattern: they initialize connections to infrastructure (Kafka, Redis) with exponential retry logic (5 attempts), enter an infinite loop, perform their specific task, and then sleep for a configured interval.

Game Catalog Producer (Release Radar) This service ensures the catalog remains current by detecting stealth releases and new store additions.

Operation Cycle: 6 Hours.

Discovery Logic (The Set Difference Strategy): The producer queries the `IStoreService/GetAppList` API, which utilizes pagination to return the complete list of $\sim 190,000$ valid App IDs on the store. It allows us to perform a precise check against our known state in Redis:

$$\Delta_{new} = \{ID_{store}\} - \{ID_{redis:scraped}\}$$

State Management: For every ID in Δ_{new} , the producer performs a full metadata scrape (Steam Store + SteamSpy). Upon successful Kafka publication, the ID is immediately added to the `steam:apps:scraped` Redis set. This makes the system self-healing; if the producer crashes during processing, it will simply rediscover the pending IDs in the next cycle.

Live Review Producer (Sentiment Stream) This producer captures the high-velocity stream of user feedback.

Operation Cycle: 60 Seconds.

The Deduplication Problem: The Steam Review API does not support a "streaming" cursor (like "give me reviews since timestamp T ") effectively due to eventual consistency issues. Instead, it offers a "Most Recent" filter. However, continuously polling "Most Recent" results in significant overlap between consecutive polls.

Sliding Window Solution: We implement a sliding window deduplication mechanism using Redis Sets.

1. **Fetch:** The producer requests the latest 100 reviews for a watched game.
2. **Filter:** It retrieves the cached set of IDs from Redis key.
3. **Publish:** Only reviews with IDs **not** in the cache are sent to Kafka.
4. **Update:** The Redis set is replaced entirely with the *current* batch of 100 IDs. A TTL (Time To Live) of 3600 seconds is applied. This prevents the Redis memory footprint from growing indefinitely while ensuring a wide enough window to catch duplicates across polls.

Player Count Producer (Heartbeat) This service polls the current concurrency metrics for the target game list.

Operation Cycle: 5 Minutes.

Stateless Polling: Unlike the other producers, this service is stateless. It queries the `ISteamUserStats` API for the current `player_count`.

Schema Compatibility: A critical design challenge in Kappa Architectures is handling schema evolution between historical backfills and live streams. The historical data contains monthly aggregates (avg, peak), while live data contains point-in-time snapshots. To maintain a single unified schema for downstream consumers, this producer explicitly injects null values for historical fields:

```
1 message = {  
2     "event_timestamp": dt.datetime.utcnow().isoformat() + "Z",  
3     "app_id": appid,  
4     "player_count": data['player_count'],  
5     "peak_players_monthly": None # Explicit NULL for schema alignment  
6 }
```

This allows the Spark job to read both historical JSONL dumps and live Kafka streams using the exact same `StructType` definition without throwing schema mismatch errors.

3.2.6 Kafka Infrastructure Configuration

The event bus is configured not just for message passing, but for long-term data persistence.

```
1 apiVersion: kafka.strimzi.io/v1  
2 kind: KafkaTopic  
3 metadata:  
4   name: steam-reviews-raw  
5   labels:  
6     strimzi.io/cluster: my-kafka-cluster  
7 spec:  
8   partitions: 10  
9   replicas: 1  
10  config:  
11    # Infinite retention for Kappa Architecture  
12    # This allows the log to serve as the permanent system of record  
13    retention.ms: -1
```

Listing 1: Kafka Topic Configuration

Three primary topics are defined, each tuned for its specific data profile:

1. `steam-games-raw` (3 Partitions): Low volume (metadata changes slowly), but high value. Three partitions allow for sufficient parallelism if the catalog grows significantly.
2. `steam-reviews-raw` (10 Partitions): High volume and high velocity. Partitioning by App ID ensures that all reviews for a specific game land on the same partition, guaranteeing strict ordering within the context of a single game.
3. `steam-player-counts-raw` (3 Partitions): High frequency time-series data. Standard partitioning provides balanced throughput for time-window aggregations.

3.3 Spark Processing Layer

The Silver Layer serves as the refinement stage, converting raw, semi-structured ingestion data into strictly typed, enriched, and deduplicated Iceberg tables. It utilizes three specialized processing modules.

3.3.1 Silver Layer: Data Cleaning & Ingestion

Player Metrics Stream (`raw_clean_history.py`) This module ingests high-frequency telemetry data from the `steam-player-counts-raw` Kafka topic.

- **Adaptive Execution Strategy:** To handle both massive historical loads and low-latency updates efficiently, the script dynamically configures triggers based on the `BACKFILL` environment variable:
 - *Backfill Mode:* Configures a watermark of "36500 days" and uses `Trigger.AvailableNow`. This allows Spark to treat the entire Kafka history as a single logical batch, maximizing throughput.
 - *Live Mode:* Sets a "1 hour" watermark with a 1-minute processing trigger to minimize latency for real-time dashboards.
- **Schema Enforcement:** Deserializes JSON payloads into a strict schema (`app_id`, `player_count`, `event_timestamp`), ensuring correct data types (`DoubleType`, `LongType`) before writing to storage.
- **Data Integrity:** Implements exactly-once semantics via `dropDuplicates` on composite keys (`app_id`, `event_timestamp`), preventing duplicate records caused by Kafka re-delivery.

Game Metadata Stream (`raw_clean_steam_apps.py`) This module handles the complex parsing of application metadata from the `steam-games-raw` topic.

- **Schema Evolution Handling:** To manage the variability of keys in the raw JSON, the pipeline uses a Map-Explode pattern. Data is first read as a Map, exploded into rows, and then parsed into a deeply nested `StructType`, preventing job failures due to schema mismatch.
- **Content Sanitization:**
 - *Text Cleaning:* A custom Regex function, `clean_text_native()`, strips HTML tags (e.g., `<br>`, `<li>`) and normalizes special entities.
 - *Standardization:* UDFs convert diverse regional age ratings (PEGI, ESRB) into a standardized integer and parse multilingual date strings into ISO format.
- **Output Target:** The enriched data is persisted to `nessie.silver.steam_games`.

Review Sentiment Stream (`raw_clean_review.py`) This module processes user-generated reviews from the `steam-reviews-raw` Kafka topic. Unlike telemetry data, review data contains complex nested structures and unstructured text.

- **Adaptive Streaming Configuration:** Similar to the player metrics stream, this module employs adaptive triggers. However, the **Live Mode** watermark is set to "6 hours" (compared to 1 hour for telemetry) to accommodate the higher latency and potential out-of-order arrival inherent in user-submitted content.
- **Transformation Logic:**
 - *Flattening:* The nested `author` struct is flattened to promote valuable metrics—such as `playtime_forever` and `num_games_owned`—to top-level columns.
 - *Text Sanitization:* The `clean_review_text` function utilizes Regex to strip URLs, HTML artifacts, and normalize whitespace in the review body.
- **Stateful Deduplication:** Ensures dataset uniqueness by applying `dropDuplicates` on the `recommendationid` key within the defined watermark window.
- **Output Target:** Appends refined data to the `nessie.silver.steam_reviews` Iceberg table.

3.3.2 Gold Layer: Aggregation & Analytics

The Gold Layer (`gold_analytics.py`) acts as the central intelligence engine. It consumes refined Silver tables and orchestrates complex transformations to derive business insights.

Functional Overview: Analytical Datasets The pipeline utilizes a micro-batch architecture to transform enriched metadata into five specialized analytical views.

1. **Game Facts Dimension (`game_fact`):** This table serves as the central source of truth for game metadata. The transformation logic performs dimensional enrichment by:
 - **Price Segmentation:** Applies conditional logic to categorize continuous pricing data into discrete buckets (e.g., "Free", "0-9.99", "50+"), facilitating categorical analysis.
 - **Playtime Calculation:** Derives a `playtime_total` metric by aggregating historical playtime (`avg_playtime_forever`) with recent activity (`avg_playtime_2weeks`).
 - **Developer Extraction:** Normalizes the developer array to extract the primary developer for simplified grouping.
2. **Genre Analytics (`game_genre`):** This dataset provides a granular view of market performance by genre. The transformation process employs an `explode_outer` operation to unpivot games with multiple genres, ensuring accurate attribution. It computes metrics at two levels:

- **Market Overview:** Calculates the total game count, average pricing, and identifies the top 5 games per genre based on revenue and engagement using a custom ranking algorithm.
 - **Developer Dominance:** Aggregates developer statistics within each genre to identify top-performing creators (e.g., the top RPG developers by positive reviews) via a broadcast join strategy.
3. **Developer Performance (`game_dev`):** This view evaluates the portfolio health of content creators. It groups data by developer to calculate key performance indicators (KPIs):
- **Portfolio Volume:** Total count of distinct applications released.
 - **Monetization Strategy:** Calculates the volume of “Free-to-Play” titles versus paid titles, alongside the average portfolio price.
 - **Reputation:** Aggregates the total positive reviews across all titles and embeds a ranked list of the developer’s top 5 highest-performing games.
4. **Price Tier Analysis (`price_segment`):** This dataset investigates the correlation between pricing strategy and user engagement. By grouping games into the computed price segments, the pipeline aggregates:
- **Engagement Metrics:** Average total playtime and average positive review counts per tier.
 - **Volume Analysis:** The total number of games competing within each price bracket.
 - **Top Performers:** Identifies the outliers (top 5 games) within each price bucket to highlight successful pricing strategies.
5. **Release Trends (`release_trend`):** This time-series dataset monitors industry velocity. It filters the dataset to a specific window (2015–2025) to analyze:
- **Market Saturation:** The total number of distinct games released per year.
 - **Developer Activity:** Utilizes a `collect_set` aggregation to track the distinct set of active developers launching titles in a given year, providing insights into creator retention and market entry.

Real-Time Windowed Analytics Complementing the metadata analysis, the pipeline executes two concurrent stateful streaming queries to process high-velocity telemetry and user sentiment data. These pipelines utilize event-time processing to derive temporal insights.

6. **Player Concurrency Trends (`player_30min`):** This pipeline monitors server load and game popularity in near real-time by processing the `steam_history` stream.
- **Windowing Strategy:** Utilizes 30-minute tumbling windows to aggregate raw telemetry data.
 - **Metric Calculation:** Computes both the **Maximum** player count (to identify peak load events) and the **Average** player count (to determine sustained engagement) within each window.

- **Late Data Handling:** Applies a 1-hour watermark strategy, allowing the system to correctly aggregate data that arrives out of order due to network latency before finalizing the window state.

7. **Temporal Sentiment Analysis (`reviews_hourly`):** This pipeline tracks the evolution of community sentiment over time by processing the `steam_reviews` stream.

- **Windowing Strategy:** Aggregates reviews into 1-hour tumbling windows based on the creation timestamp.
- **Sentiment Derivation:**
 - Converts boolean voting data (`voted_up`) into aggregate counts of positive and negative reviews.
 - Derives **Sentiment Ratios** (Positive vs. Negative percentage) to detect sudden shifts in community opinion (e.g., "review bombing" events).
- **Quality Tracking:** Aggregates the `weighted_vote_score` to assess the average quality and helpfulness of reviews submitted during that hour.
- **State Management:** Configured with a 2-hour watermark to accommodate significant delays in review ingestion.

Technical Implementation & Spark Optimization To meet strict performance requirements and handle complex logic within a streaming context, the Gold Layer employs several advanced Spark techniques.

Complex Aggregations and Advanced Transformations The system avoids expensive window functions for ranking, which are often performance-prohibitive in streaming.

- **Custom Top-K Aggregation:** A custom function, `calculate_top_k_in_group`, packs sorting metrics and metadata into a struct. It then utilizes Spark SQL functions to sort the array and slice the top k elements within the aggregation phase itself.
- **Unpivot Operations:** The `transform_genre_analytics` function uses `explode_outer` to normalize the one-to-many relationship between games and genres, allowing for accurate granular aggregation.
- **Chaining:** Complex business logic, such as price binning (`get_price_segment_col`), is implemented via chained `when/otherwise` expressions, enabling lazy evaluation and optimizer fusion.

Performance Optimization Strategies

- **Caching and Persistence:** The script applies `persist (StorageLevel.MEMORY_AND_DISK)` to the `facts_batch` DataFrame. Since a single incoming batch fuels five distinct downstream transformations, caching ensures the base cleaning logic is executed only once per batch, significantly

reducing I/O and CPU overhead. A `finally` block guarantees `unpersist()` is called to manage memory.

- **ForeachBatch Execution:** By utilizing a single `foreachBatch` sink, the system manages multiple write targets (Iceberg and MongoDB) within a single streaming handle, reducing the overhead of maintaining independent offsets for every analytical view.

Streaming Processing and State Management

- **Watermarking:** To handle late-arriving telemetry data, the system applies watermarking (`.withWatermark("event_timestamp", "1 hour")`). This ensures the state store retains intermediate aggregation data long enough to account for network delays before dropping the state.
- **Window Functions:** Tumbling windows are used for time-series analysis:
 - *Player Stats:* 30-minute tumbling windows calculate max/avg concurrency.
 - *Reviews:* 1-hour tumbling windows track sentiment trends (positive/negative ratios).
- **Fault Tolerance:** Dedicated HDFS checkpoint directories are configured for each stream, ensuring exactly-once processing guarantees across application restarts.

Export Layer: The Dual Sink Strategy To support both Historical Analysis (OLAP) and Real-Time Serving (OLTP), the `gold_analytics.py` module implements a custom "Dual Sink" pattern.

- **Mechanism:** The `write_dual_sink` function is invoked at the conclusion of every transformation task.
- **Sink 1 (Data Lake):** Appends data to Nessie-managed Iceberg tables in HDFS. This enables time-travel queries and long-term historical retention.
- **Sink 2 (Operational DB):** Pushes aggregated metrics to MongoDB collections. This allows the application frontend to query pre-computed dashboards with low latency, without scanning the file system.

3.4 Distributed Learning Pipelines with Spark ML

This section presents two distributed machine learning pipelines implemented using Apache Spark ML, addressing large-scale regression and recommendation problems. Spark ML is selected due to its native support for distributed data processing, its clear separation between transformation and estimation stages, and its pipeline abstraction, which enables reproducible and production-ready learning workflows. Both pipelines operate directly on versioned Iceberg datasets and persist trained models to a distributed storage system for subsequent inference.

Data preprocessing is performed using Spark SQL operations. Missing values are handled through null coalescing, boolean attributes are converted into numerical representations, and categorical attributes such as game genres and categories are represented as arrays of strings. The two paragraphs below discuss details about the pipeline spark ML for two machine learning problems in Spark ML.

3.4.1 Regression Pipeline for Game Playtime Prediction

The regression task aims to predict the average lifetime playtime of a game based on a heterogeneous set of structured attributes. The learning workflow is implemented as a Spark ML Pipeline, in which feature extraction and model estimation are explicitly organized into sequential stages.

The regression pipeline consists of the following stages:

1. **Genre Vectorization (Estimator).** A `CountVectorizer` estimator is applied to the array-based genre attribute. During pipeline fitting, this estimator scans the distributed training dataset to learn a vocabulary and generates sparse vector representations for genre information.
2. **Category Vectorization (Estimator).** A second `CountVectorizer` estimator is independently applied to the category attribute. This separation allows the pipeline to capture distinct semantic structures associated with different categorical feature groups.
3. **Feature Assembly (Transformer).** A `VectorAssembler` transformer aggregates numerical attributes, boolean indicators, and vectorized categorical features into a single feature vector. This stage performs a deterministic transformation without learning additional parameters.
4. **Regression Model Estimation (Estimator).** A `RandomForestRegressor` serves as the final estimator in the pipeline. During fitting, it learns an ensemble of decision trees in a distributed manner, modeling non-linear relationships between the assembled features and the target playtime variable.

Invoking the `fit` operation on the pipeline sequentially executes all estimator stages and produces a unified `PipelineModel`. This model encapsulates both learned feature transformations and the trained regression model, ensuring consistent preprocessing and inference during evaluation and deployment.

3.4.2 Recommendation Pipeline for Implicit Feedback Modeling

The recommendation task is formulated as a collaborative filtering problem based on implicit user feedback. Instead of explicit ratings, user engagement signals are transformed into a continuous confidence score that reflects varying degrees of preference.

The recommendation pipeline is constructed as a sequence of estimation stages designed to prepare interaction data for matrix factorization:

1. **User Identifier Indexing (Estimator).** A `StringIndexer` estimator maps raw user identifiers to contiguous integer indices. This stage performs a distributed scan over the interaction dataset to

construct a consistent user index required by the recommendation model.

2. **Item Identifier Indexing (Estimator).** A second `StringIndexer` estimator maps game identifiers to contiguous integer indices. Together with user indexing, this stage ensures that the interaction matrix is represented in a compact numerical form.
3. **Implicit Feedback Model Estimation (Estimator).** The final stage employs the Alternating Least Squares (ALS) estimator configured for implicit feedback. During fitting, ALS decomposes the sparse user-item interaction matrix into low-dimensional latent factor representations by alternately optimizing user and item factors in a distributed fashion.

By embedding both identifier indexers and the ALS model within a single Spark ML Pipeline, the resulting `PipelineModel` jointly stores index mappings and learned latent factors. This design guarantees that identifier consistency is preserved during large-scale recommendation generation and subsequent inference.

The two pipelines demonstrate how Spark ML enables the structured composition of distributed learning workflows through explicit estimator-based pipeline stages. The regression pipeline integrates ensemble learning with structured and categorical features, while the recommendation pipeline leverages implicit feedback and matrix factorization for personalized recommendation. Together, these pipelines illustrate a scalable and reproducible approach to machine learning in Big Data systems.

3.5 Monitoring and Quality Assurance

Ensuring the reliability of a streaming data pipeline requires both proactive data quality enforcement and reactive system observability. This section describes our approach to both.

3.5.1 Data Quality Strategy: Quarantine and Audit

The pipeline implements a "Post-Write Verification" pattern, prioritizing data availability while ensuring long-term consistency. Instead of a blocking "Promotion Gate", we utilize a reactive Quarantine mechanism combined with auditing.

The Quarantine Process (Spark SQL) The data quality jobs (`dq_game.py`, `dq_review.py`) execute a multi-step remediation workflow:

1. **Identification:** Bad records (duplicates, missing keys, schema violations) are identified using native Spark SQL expressions for maximum performance.
2. **Isolation:** Identified records are moved to a dedicated `nessie.quarantine` catalog (e.g., `steam_reviews_bad`) for manual inspection.

3. **Remediation:** The system executes atomic `DELETE` operations against the production Silver tables to physically remove the invalid data, ensuring downstream consumers (Gold Layer) see only clean data.

Auditing with PyDeequ After the quarantine process cleans the dataset, **PyDeequ** is invoked on the remaining valid data to generate verification metrics.

- **Purpose:** While utilizing Spark SQL for cleaning, PyDeequ is retained solely for generating a historical audit trail.
- **Metric Persistence:** Verification results (compliance percentages, distinctness ratios) are stored in `nessie.dq_reports` to visualize data quality trends over time in Grafana.

3.5.2 System Observability

The serving layer implements comprehensive monitoring using Prometheus for metrics collection and Grafana for visualization. This monitoring infrastructure is configured through multiple files in the `monitoring/` directories.

Prometheus Metrics Collection (monitoring/prometheus-configmap.yaml) The `prometheus.yml` file configures scraping behavior of Prometheus. The configuration defines:

Global Settings:

- `scrape_interval`: 15-second default metric collection frequency.

Scrape Configurations:

- **prometheus job:** Self-monitoring on `localhost:9090` for scrape health and cardinality tracking.
- **mongodb job:** Scrapes `mongodb-exporter.database.svc.cluster.local:9216` every 15 seconds (global default) for database metrics across namespaces.
- **kafka job:** Scrapes `kafka-exporter:9308` every 15 seconds (global default) for Kafka cluster metrics.

Prometheus stores these time-series metrics in its local time-series database, which Grafana queries for dashboard visualizations.

Grafana Data Sources (monitoring/provisioning/grafana-datasource-configmap.yaml)

The `grafana-datasource-configmap.yaml` file configures two data sources that Grafana uses to populate dashboards:

MongoDB Datasource:

- **Name:** MongoDB.
- **Type:** haohanyang-mongodb-datasource (MongoDB integration).
- **URL:** `mongodb://my-mongodb.database.svc.cluster.local:27017`.
- **Access:** Proxy mode.
- **Default:** Set as the default datasource.
- **Purpose:** Directly queries the `steam_analytics` database to visualize processed game data, player trends, and review sentiment metrics.

Prometheus Datasource:

- **Name:** Prometheus.
- **Type:** `prometheus` (time-series metrics database).
- **URL:** `http://prometheus.monitoring.svc.cluster.local:9090`.
- **Access:** Proxy mode (Grafana queries on behalf of browser).
- **Purpose:** Queries infrastructure metrics, exporter health, and pipeline performance metrics from the monitoring stack.

The dual-datasource approach enables dashboards to cross-reference infrastructure performance (from Prometheus) with the actual game-related data stored in the gold-tier (from MongoDB).

Grafana Dashboards (`monitoring/provisioning/dashboards/*.json`) Three specialized dashboard JSON files define the monitoring UI:

Dashboard: MongoDB Metrics

- **Datasource:** Prometheus
- **Description:** Show MongoDB connection pool and operational metrics

Dashboard: Steam Lens

- **Datasource:** MongoDB
- **Description:** Show a lot of insights about Steam Games such as popular genres, most played games, game releases trend, etc

Dashboard: Game Inspect

- **Datasource:** MongoDB
- **Description:** Real-time stats of player count and reviews of games

Dashboard: Dev Inspect

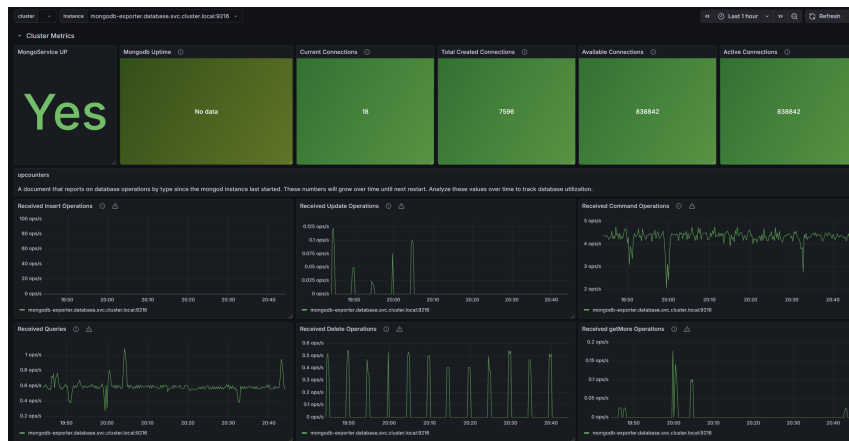


Figure 3: MongoDB Metrics Dashboard

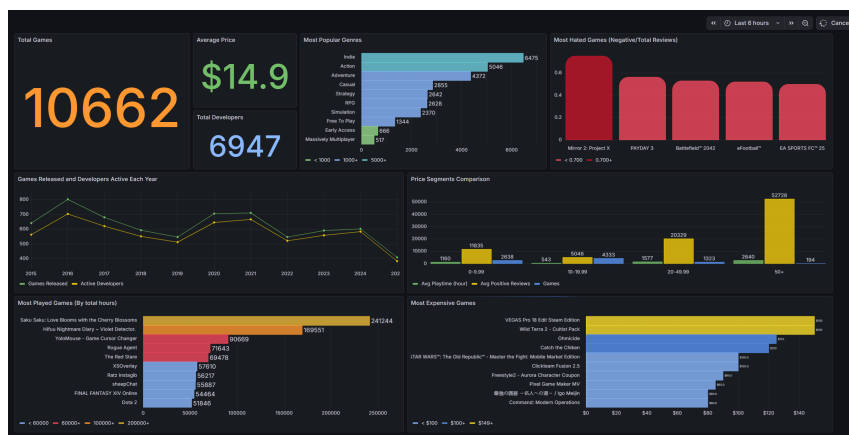


Figure 4: Steam Lens Dashboard

- **Datasource:** MongoDB
- **Description:** Choose a developer to get insights about their games (total games, average price, most popular games, etc...)

Dashboard Provisioning (monitoring/provisioning/dashboard-provider-configmap.yml)

The dashboard-provider.yml file configures automatic dashboard loading:

- **Provider Type:** File-based provisioning (dashboards loaded from /etc/grafana/provisioning/dashboards/).
- **Update Behavior:** updateIntervalSeconds: 15 checks for file changes every 10 seconds.
- **Editable:** Dashboards can be modified in UI; changes persisted to JSON files.
- **Folder Organization:** All dashboards placed in "Steam Analytics" folder.

This file-based approach enables version control of dashboard definitions and automated deployment through Docker volumes.



Figure 5: Game Inspect Dashboard



Figure 6: Dev Inspect Dashboard

Monitoring Data Flow The complete monitoring pipeline operates as follows:

1. **Metric Generation:** ; MongoDB exporter queries database statistics ,Kafka exporter queries cluster metrics.
2. **Metric Collection:** Prometheus scrapes all targets based on `prometheus.yml` configuration, storing time-series data locally.
3. **Dashboard Queries:** Grafana queries Prometheus for infrastructure metrics using PromQL; Grafana queries MongoDB using MongoDB datasource plugin with JSON expressions.
4. **Visualization:** Dashboard panels transform query results into graphs, tables, and gauges; auto-refresh intervals trigger periodic re-queries.
5. **User Interaction:** Users filter dashboards using template variables (game,developer); variable changes trigger dashboard-wide query updates.

This monitoring architecture provides real-time visibility into both infrastructure health (response times, connection pools) and business analysis metrics (player count trend, popular genres).

4 Lessons Learned

4.1 Lesson 1: Strategic Planning

4.1.1 Problem Description

- **Context and background:** The course requires building an end-to-end big data pipeline to solve a real-world problem.
- **Challenges encountered:** Our initial plan did not align with the course scope; the data sources originally selected failed to provide the high-velocity signals necessary for a functional streaming implementation.
- **System impact:** Despite completing initial development and refinement phases, the lack of real-time data forced a total project restart to satisfy the requirement for an end-to-end streaming pipeline.

4.1.2 Approaches Tried

- **Approach 1:** Attempting to refine and "patch" the existing system to force static data into a real-time framework to preserve previous work.
- **Approach 2:** Scrapping the initial roadmap to rebuild the plan from scratch, adopting a native **Kappa Architecture** and prioritizing data sources (Steam Web API, Reviews) capable of high-frequency updates.
- **Trade-offs of each approach:** Approach 1 saved initial time but failed to meet the technical criteria for real-time processing. Approach 2 required significant extra effort but ensured the system satisfied the architectural requirements for high-velocity data ingestion.

4.1.3 Final Solution

- **Detailed solution:** Changed the data sources to the Steam Web API and Steam Storefront to align precisely with the course's streaming goals.
- **Implementation details:** Built a new pipeline utilizing the `GetAppList` endpoint for the Game Catalog (Batch/Slow Stream) and the Reviews API for the Sentiment Stream (Fast Stream).
- **Metrics and results:** The new strategy successfully enabled 5-minute tumbling windows and near real-time sentiment analysis for thousands of active titles.

4.1.4 Key Takeaways

- **Technical insights:** Success in a big data project depends on thorough planning before implementation; the selected data must inherently support the architectural model's goals.
- **Best practices:** Establish a clear data requirement during the design phase to ensure that ingestion sources can provide the necessary metrics for the processing layer.

4.2 Lesson 2: The "Small Files" Problem in Streaming

4.2.1 Problem Description

- **Context:** Spark Structured Streaming triggers every 1-5 minutes to minimize latency.
- **Challenge:** Writing Parquet files directly to HDFS at this frequency generated thousands of KB-sized files per hour.
- **Impact:** This "Small Files Problem" overwhelmed the HDFS NameNode memory and degraded query performance for downstream consumers (Trino), which had to open too many file handles.

4.2.2 Solution

- **Adoption of Apache Iceberg:** We migrated the storage format from raw Parquet to Iceberg tables.
- **Outcome:** Iceberg abstracts the file layout from the reader. It supports asynchronous compaction actions that rewrite small files into larger, optimized storage files without blocking concurrent reads, maintaining high query performance on top of a streaming write pattern.

4.3 Lesson 3: Optimization Strategies in Spark Structured Streaming

4.3.1 Problem Description

- **Context:** The Gold Layer (`gold_analytics.py`) required calculating 5 distinct analytical views (Genre, Developer, Price Tiers, etc.) from the same refined data stream.
- **Challenges:** Running separate `StreamingQueries` for each view caused massive resource contention on our limited hardware (cluster of laptops). The overhead of managing multiple Kafka consumer groups and Spark orchestration threads led to driver latency exceeding 2 minutes.
- **System impact:** Real-time dashboards lagged significantly behind live events, defeating the purpose of a streaming architecture.

4.3.2 Approaches Tried

- **Approach 1:** Launching independent Spark jobs for each metric. This proved too resource-intensive, quickly consuming all available executor cores.
- **Approach 2:** Implementing a "Single-Pass" processing strategy using advanced Spark techniques.

4.3.3 Final Solution

- **Micro-Batch Caching:** We utilized `df.persist()` on the incoming micro-batch. This ensures that the costly deserialization and cleaning logic is executed only once per batch, rather than re-computed for every downstream aggregation.
- **ForeachBatch Sink:** We implemented a custom `foreachBatch` function that contains the logic to update all 5 analytical views and write them to both storage destinations (Iceberg and MongoDB) within a single execution graph.
- **Results:** This architecture reduced CPU load by approximately 40% and brought end-to-end latency down from minutes to seconds.

4.3.4 Key Takeaways

- **Technical insights:** In resource-constrained environments, minimizing data movement and re-computation is critical.
- **Best practices:** Use `foreachBatch` to multiplex multiple write sinks from a single stream, preventing the overhead of separate job submissions.

4.4 Lesson 4: Resource Management on Commodity Hardware

4.4.1 Problem Description

- **Context:** The entire infrastructure (Kafka, Spark, Mongo, MinIO) runs on a self-managed K3s cluster hosted on personal laptops connected via Tailscale.
- **Constraint:** Unlike cloud environments where vertical scaling is trivial, our hardware had hard physical limits on RAM and CPU.
- **Challenge:** Initial attempts at dynamic scaling caused cluster instability, where aggressive executor allocation starved critical system components (Zookeeper) of resources.

4.4.2 Solution

- **Static Resource Partitioning:** We pivoted from dynamic autoscaling to strict static resource quotas. By explicitly defining minimal container sizes (e.g., `instances: 2, memory: "1g"` for backfills), we ensured that heavy batch jobs could run continuously without exceeding the cluster's safe capacity.
- **Distinct Scheduling Profiles:** We architected separate configurations for streaming jobs (low-latency, single executor) and backfill jobs (throughput-optimized, dual executors), preventing sporadic batch processing demands from monopolizing cluster resources needed by the real-time pipeline.

4.4.3 Key Takeaways

- **Technical insights:** On highly constrained edge hardware, the scheduling overhead of dynamic orchestration can outweigh its benefits.
- **Best practices:** Deterministic static allocation provides superior stability guarantees compared to autosensing when the total resource pool is small and fixed.